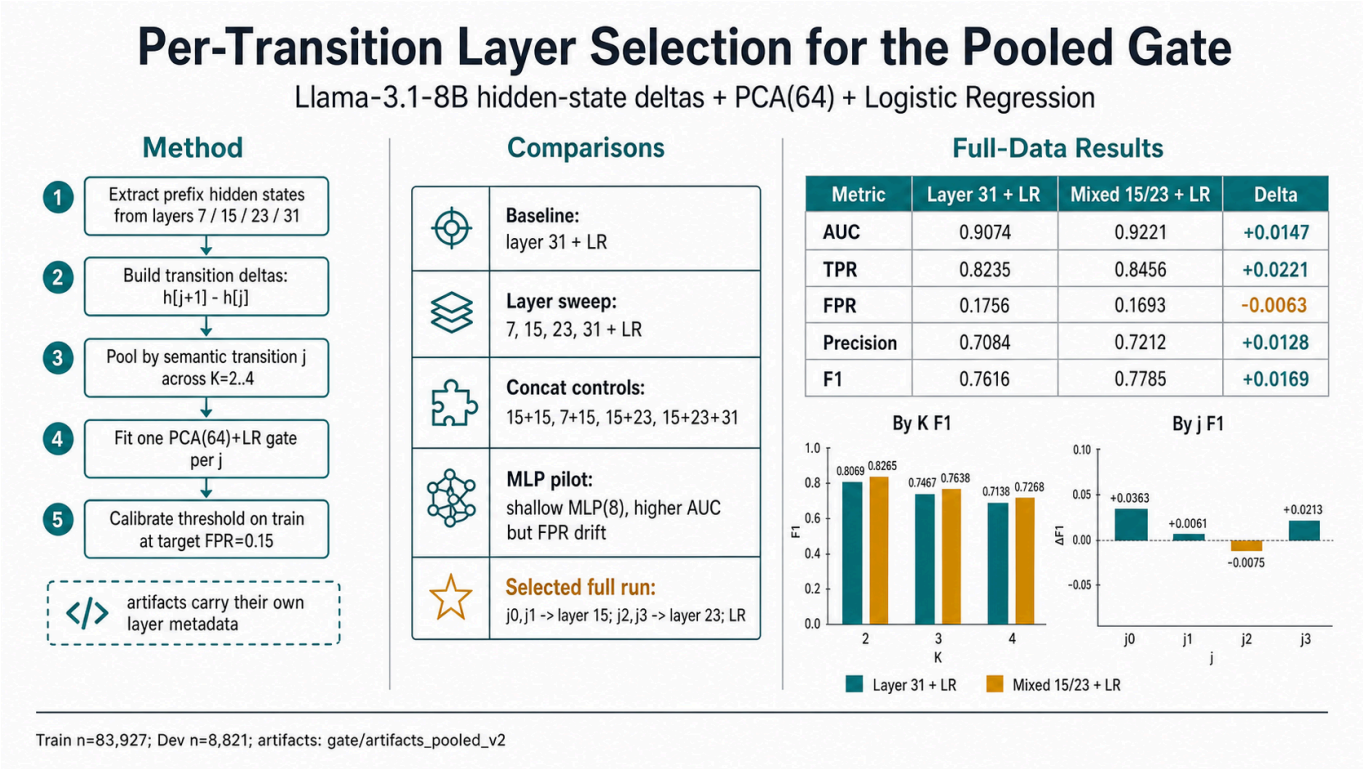


0705 update — gate 层选择 / 分类器 pilot 实验

记录这次 gate 相关的探索性实验结论，供后续继续跟进。全量数据验证已经跑完，结论成立——见下面 "全量数据验证结果（最终结论）" 一节。再往下的"pilot 子采样实验"是这个结论最初是怎么摸出来的过程记录（层扫描、concat 对照组、MLP 对比等），数字都是 pilot 规模，仅供参考方法论，不代表最终效果。



背景

现有生产 gate (gate/artifacts_pooled/) 固定用 Llama-3.1-8B-Instruct 的最后一层 (layer 31) + PCA(64)+LogisticRegression，四个 pooled transition (j=0..3) 共享同一层。这次想验证：

1. 最后一层是不是最优选择？
2. 能不能通过更好的特征/分类器组合，缓解已知的"K=4（深层 transition）判别力偏弱"问题（production dev 指标里 K=4 的 F1 明显低于 K=2）？

全量数据验证结果（最终结论）

pilot 子采样阶段（见下文）摸出的最优方案——j=0,1 用 layer 15、j=2,3 用 layer 23，分类器还是原来的 LogisticRegression（不用 MLP，避免下面提到的阈值校准问题）——已经在**全量 train（83927 条，不是 pilot 的 20615 条子采样）+ 全量 dev（8821 条）**上重新跑过 fit_lr_gate_pooled.py --per-j-layers 0:15,1:15,2:23,3:23，结论成立，而且是全面提升（不是拿 FPR 换 TPR 这种取舍）：

	AUC	TPR	FPR	Precision	F1
31 + LR（生产现状，全量）	0.9074	0.8235	0.1756	0.7084	0.7616

	AUC	TPR	FPR	Precision	F1
per-j 混合(15,15,23,23) + LR (全量)	0.9221	0.8456	0.1693	0.7212	0.7785
差值	+0.0147	+0.0221	-0.0063 (更低更好)	+0.0128	+0.0169

分 j：

j	31 F1	混合 F1	差值
j=0 (Q→E1)	0.7513	0.7876	+0.0363
j=1 (E1→E2)	0.7699	0.7760	+0.0061
j=2 (E2→E3)	0.7610	0.7535	-0.0075 (唯一变差, 幅度很小)
j=3 (E3→E4)	0.7843	0.8056	+0.0213

分 K (K=4 之前一直是短板)：

K	31 F1	混合 F1	差值
K=2	0.8069	0.8265	+0.0196
K=3	0.7467	0.7638	+0.0171
K=4	0.7138	0.7268	+0.0130

跟 pilot 阶段的 MLP 版本 (AUC 更高但 FPR 校准跑偏到 0.2348) 不同, **这次用的还是纯 LR**, FPR (0.1693) 比生产现状 (0.1756) 还略低一点, 说明阈值校准也是准的, 没有 MLP 那个问题。

产出物：

- 数据：`hidden_states/pilot_multilayer/` (`train_pilot/` 现在是全量 83927 条, `dev/` 全量 8821 条, 7/15/23/31 四层都在, 同一份多层 npz 存储, 不用再单独维护一份只含 31 层的目录——见 `gate/fit_lr_gate.py::collect_split` 的格式自动识别逻辑)。
- 训练产出：`gate/artifacts_pooled_v2/` (j0~j3.joblib + meta.json, 每个 joblib 自带 "layer" 字段记录自己实际用的层) + `gate/results_pooled_v2/` (dev 指标, 即上表数字的来源)。
- 支持这一切的代码改动：
 - `gate/fit_lr_gate.py::collect_split` 新增 `layer` 参数, 自动识别 npz 是生产单层格式还是 pilot 多层格式。
 - `gate/fit_lr_gate_pooled.py` 新增 `--per-j-layers` (每个 j 可以指定不同层, 各自的 joblib 记录自己用的层) 和 `--train-split-name` (因为 pilot 目录叫 `train_pilot`, 生产目录叫 `train`)。
 - `retrieval/run_retrieval_exp_wavefront.py` : `batch_last_hidden` 从"只读一层"改成"一次 forward 同时读出所有需要的层"; `score_gate_requests` 不再用一个全局 `--layer`, 而是对每个请求读它自己 (K,j) 对应 artifact 里的 "layer" 字段, 决定用哪层的 hidden state 打分——per-j 混层这件事从头 到尾都是"artifact 自己携带的元数据"驱动的, 检索脚本里没有写死"第几跳用第几层"。

还没做的事：`artifacts_pooled_v2` 目前只是训出来放着，还没有：

1. 正式替换成生产 `gate/artifacts_pooled/` ；
2. 拿去跑一次端到端的 `retrieval/run_retrieval_exp_wavefront.py`，看实际检索/答案指标（EM/F1/ recall）有没有跟着涨——目前的验证都停留在 `gate` 自身的 TPR/FPR/F1，还没有验证对下游检索任务的 实际影响。

pilot 子采样实验（探索过程记录）

下面的内容是全量结论出来之前、在 `pilot` 子采样上做的一系列探索——记录了怎么从"最后一层"一步步摸到"per-j 混层"这个方案的过程（层扫描、concat 对照组排除维度假象、MLP 对比、阈值校准问题的发现），方法论上的参考价值大于具体数字（数字是 `pilot` 规模，跟上面的全量结论不完全一致）。

实验设置

- 新脚本（不影响生产代码）：
- `hidden_states/extract_hidden_states_multilayer_pilot.py` — 一次 forward 同时读出多层 hidden state（`output_hidden_states=True` 本来就会算出全部 32 层，只是生产代码只读了第 31 层）。支持增量写盘+内存驱逐、按 token 预算自适应 batch size、`--resume`。
- `gate/compare_layers_pilot.py` — 读上面的多层 npz，支持单层对比 / 多层拼接（concat） / 按 transition 混合选层（per-j-layers） / 切换分类器（LR 或浅层 MLP）。
- 数据：
- train：分层子采样，K=4（最稀缺、也是 pooled j=3 唯一的数据来源）**全量保留**（2023 条/桶），K=2/K=3 各桶 cap 到 1500 条（这两个 K 本身就比较 K=4 丰富得多，且同时服务多个 pooled j，采样减量对结果影响较小）。共 20615 条。
- dev：全量 8821 条，未采样。
- 分类器统一在 PCA(64) 输出的 64 维特征上训练（跟生产环境 pipeline 一致），只换 PCA 之后接的 模型或者喂给 PCA 的原始层。

结果汇总

配置	j=0 (Q→E1)	j=1 (E1→E2)	j=2 (E2→E3)	j=3 (E3→E4)	overall
31 + LR（生产环境现状）	0.9264	0.8925	0.8531	0.8674	0.9007
7 + LR	0.7814	0.7683	0.6970	0.7536	0.7589
15 + LR	0.9497	0.9005	0.8338	0.8739	0.9151
23 + LR	0.9377	0.9037	0.8465	0.8850	0.9113
concat(15+15) + LR（对照组，验证"是否只是维度多了就好"）	0.9494	0.9010	0.8343	0.8736	0.9152
concat(7+15) + LR	0.9479	0.9011	0.8375	0.8798	0.9145
concat(15+23) + LR	0.9434	0.9004	0.8513	0.8911	0.9147
concat(15+23+31) + LR	0.9262	0.8929	0.8543	0.8677	0.9010

配置	j=0 (Q→E1)	j=1 (E1→E2)	j=2 (E2→E3)	j=3 (E3→E4)	overall
15 + MLP(8)	0.9529	0.9113	0.8523	0.8738	0.9254
23 + MLP(8)	0.9415	0.9104	0.8604	0.9004	0.9205
concat(15+23) + MLP(8)	0.9479	0.9122	0.8537	0.8943	0.9238
per-j 混合(0,1→15; 2,3→23) + MLP(8)	0.9529	0.9113	0.8604	0.9004	0.9266

MLP 统一是单隐层、8 个神经元、ReLU 激活 (`sklearn.neural_network.MLPClassifier`)，刻意做得很浅，避免"结论到底是 trace 信号本身有效，还是分类器表达能力堆出来的"这个问题（呼应一直以来"gate 保持 简单"的设计取向）。

混淆矩阵指标（TPR / FPR / Precision / F1）

阈值是训练集上按 `target_fpr=0.15` 校准出来的（跟生产 `fit_lr_gate_pooled.py` 同一套配方）。只有下面这几行是从完整 JSON 里读出来的，其余配置

(`concat(15+15)/concat(7+15)/concat(15+23+31)/15+MLP/23+MLP/concat(15+23)+MLP`) 当时只保存了终端打印的 AUC 表格，`layer_compare_metrics.json` 每次跑都会被覆盖，所以这几个的 TPR/F1 现在已经拿不到了，需要的话得重新跑一遍。

配置	TP	FP	FN	TN	TPR	FPR	Precision	F1
31 + LR（生产环境现状，pilot 规模）	5340	2464	1064	9902	0.8339	0.1993	0.6843	0.7517
7 + LR	3700	2516	2704	9850	0.5778	0.2035	0.5952	0.5864
15 + LR	5487	2451	917	9915	0.8568	0.1982	0.6912	0.7652
23 + LR	5425	2481	979	9885	0.8471	0.2006	0.6862	0.7582
concat(15+23) + LR	5464	2452	940	9914	0.8532	0.1983	0.6902	0.7631
per-j 混合(0,1→15; 2,3→23) + MLP(8)	5758	2903	646	9463	0.8991	0.2348	0.6648	0.7644

需要提醒一点：per-j 混合+MLP 这一行虽然 AUC 最高、TPR 也最高（0.8991，比生产现状的 0.8339 高不少，漏检更少），但实际 FPR 是 **0.2348**，明显高于校准目标的 **0.15**——比生产现状的 0.1993 还要高，说明这套阈值校准（在 train 上校准、拿到 dev 上用）对 MLP 不像对 LR 那么准。可能的原因：

LogisticRegression 的输出概率天然比较"平滑"、train/dev 之间迁移得好；MLP 更灵活，train 上校出来的阈值不一定能在 dev 上稳定复现同样的 FPR。这意味着如果真的要 ML 用到生产环境，**阈值校准这一步需要重新设计或者更谨慎地验证**（比如在校准时也考虑早停/交叉验证得到的 dev-like 数据），不能直接照搬现在 LR 的校准方法。AUC 层面的提升是真实的，但"卡在某个阈值上到底触发多少次"这件事，MLP 版本目前还没有调好。

关键结论

1. 最后一层 (31) 不是最优层。中间层 (15、23) 在同等训练规模下整体 AUC 都比 31 高 1~1.5 个点。
2. 不同层擅长不同 **transition** : 15 在浅层 ($j=0/j=1$, 问题刚展开时) 更强; 23 在深层 ($j=2/j=3$, 对应 $K=3/K=4$ 更难的跳数) 更强——且 23 在 $j=2/j=3$ 上单独就是所有配置里最好的, 比 `concat(15+23)` 还高。
3. "维度多了就好"这个假象被排除了: `concat(15+15)` 这个对照组 (同一层跟自己拼) 几乎跟单独一层完全一样 (0.9152 vs 0.9151), 说明单纯堆维度没用; 只有真正拼接**不同层**才有提升, 且提升集中在两层各自互补的地方 (深层 transition)。
4. 加入 31 进 **concat** 反而拖累 (`concat(15+23+31)` overall 掉到 0.901, 跟 31 单独差不多), 说明 31 信息质量偏弱, 混进去会稀释 15/23 的优势。推测原因: 最后一层被"预测下一个 token"这个训练目标塑形得最厉害, 可能把一部分"这段是否连贯自洽"的抽象语义信号压缩掉了; 23 这种中后段的层还没被这个压力主导, 反而保留得更完整。
5. 浅层 **MLP (1隐层+ReLU)** 比纯 **LR** 有稳定但温和的提升, 在所有测试配置上都一致地涨 (不是只有某一个 transition 涨), 且是在没参与训练的 dev 集上量出来的, 不太像是过拟合噪声的假象。
6. 目前最好的方案: 按 transition 分别选层 ($j=0,1$ 用 15; $j=2,3$ 用 23) + 浅层 MLP, 而不是 "一刀切用同一层"或者"把两层硬拼在一起"。pooled 协议本来就是给每个 j 训练独立模型, 没有规定它们必须共享同一层特征, 所以这个混合方案在架构上是自然的, 不是额外的复杂度。
 - overall AUC : 0.9007 (生产现状) → **0.9266** (+0.026)
 - $j=3$ ($K=4$ 最后一跳, 此前一直是短板) : 0.8674 → **0.9004** (+0.033)
7. 但 **MLP** 版本的阈值校准还没调好: per- j 混合+MLP 在校准目标 $FPR=0.15$ 下, dev 上实际测出来 $FPR=0.2348$, 比生产现状的 0.1993 还高——AUC/排序质量的提升是真的, 但"这个阈值到底该定在哪"这件事, 现在直接套用 LR 那套校准方法对 MLP 不够准, 换生产之前要单独解决。

下一步

全量验证已经做完 (见上面"全量数据验证结果"), 结论成立。剩下的事:

1. 决定要不要把 `gate/artifacts_pooled_v2/` 转正, 替换生产 `gate/artifacts_pooled/`——这是个 人为决策点, 不是技术问题: 全量 gate 指标 (TPR/FPR/F1) 已经确认全面变好, 但还没有验证过对 **下游检索任务** (recall/EM/F1) 的实际影响。
2. 跑一次端到端 `retrieval/run_retrieval_exp_wavefront.py`, 用 `artifacts_pooled_v2` 对比现在生产 `artifacts_pooled` 的检索效果——这是唯一还没跑过的验证环节, 也是决定要不要转正之前最后一块拼图。检索脚本已经改好支持按 artifact 自带的 "layer" 元数据自动分层打分, 不需要再改代码, 直接把 `--artifacts-dir` 指向 `artifacts_pooled_v2` 就能跑。
3. **MLP 版本暂时搁置**: pilot 阶段 MLP 比纯 LR 的 AUC 还要高一点 (0.9266 vs 0.9147, pilot 规模), 但阈值校准跑偏的问题 (FPR 0.2348 远超目标 0.15) 没解决, 所以这次全量验证特意选了纯 LR 的 per- j 混层方案, 没有引入 MLP。如果以后还想榨这部分潜力, 需要先重新设计 MLP 的阈值校准方式 (比如 校准时用早停切出来的验证集, 而不是全部训练数据), 再重新做一次全量验证。
4. 可选的深挖方向 (不急): 在 23 和 31 之间再测几个层 (比如 27、29), 看深层 transition 的 "甜点"具体在哪一层, 进一步理解这个"深度随 hop 数变化"的现象。